

Hands-on Lab: String Patterns, Sorting and Grouping in MySQL

Estimated time needed: 30 minutes

In this lab, you will learn how to create tables and load data in the MySQL database service using the phpMyAdmin graphical user interface (GUI) tool.

Objectives

After completing this lab, you will be able to:

- Filter the output of a SELECT query by using string patterns, ranges, or sets of values.
- Sort the result set in either ascending or descending order in accordance with a pre-determined column.
- Group the outcomes of a query based on a selected parameter to further refine the response.

Software Used in this Lab

In this lab, you will use [MySQL](#). MySQL is a Relational Database Management System (RDBMS) designed to efficiently store, manipulate, and retrieve data.



To complete this lab you will utilize MySQL relational database service available as part of IBM Skills Network Labs (SN Labs) Cloud IDE. SN Labs is a virtual lab environment used in this course.

Database Used in this Lab

The database used in this lab is an internal database. You will be working on a sample HR database. This HR database schema consists of 5 tables called **EMPLOYEES**, **JOB_HISTORY**, **JOBS**, **DEPARTMENTS** and **LOCATIONS**. Each table has a few rows of sample data. The following diagram shows the tables for the HR database:

SAMPLE HR DATABASE TABLES

EMPLOYEES

EMP_ID	F_NAME	L_NAME	SSN	B_DATE	SEX	ADDRESS	JOB_ID	SALARY	MANAGER_ID	DEP_ID
E1001	John	Thomas	123456	1976-01-09	M	5631 Rice, OakPark,IL	100	100000	30001	2
E1002	Alice	James	123457	1972-07-31	F	980 Berry Ln, Elgin,IL	200	80000	30002	5
E1003	Steve	Wells	123458	1980-08-10	M	291 Springs, Gary,IL	300	50000	30002	5

JOB_HISTORY

EMPL_ID	START_DATE	JOBS_ID	DEPT_ID
E1001	2000-01-30	100	2
E1002	2010-08-16	200	5
E1003	2016-08-10	300	5

JOBS

JOB_ID	JOB_TITLE	MIN_SALARY	MAX_SALARY
100	Sr. Architect	60000	100000
200	Sr.SoftwareDeveloper	60000	80000
300	Jr.SoftwareDeveloper	40000	60000

DEPARTMENTS

DEPT_ID	DEPT_NAME	MANAGER_ID	LOC_ID
2	Architect Group	30001	L0001
5	Software Development	30002	L0002
7	Design Team	30003	L0003

LOCATIONS

LOC_ID	DEPT_ID
L0001	2
L0002	5
L0003	7

Load the database

Using the skills acquired in the previous modules, you should first create the database in MySQL. Follow the steps below:

- Open the phpMyAdmin interface from the Skills Network Toolbox in Cloud IDE.
- Create a blank database named 'HR'. Use the script shared in the link below to create the required tables.
[Script_Create_Tables.sql](#)
- Download the files in the links below to your local machine (if not already done in previous labs).
[Departments.csv](#)
[Jobs.csv](#)
[JobsHistory.csv](#)
[Locations.csv](#)
[Employees.csv](#)
- Use each of these files to the interface as data for respective tables in the 'HR' database.

String Patterns

You can use string patterns to filter the response of a query. Let's look at the following example:

Say you need to retrieve the first names F_NAME and last names L_NAME of all employees who live in Elgin, IL. You can use the LIKE operator to retrieve strings that contain the said text. The code will look as shown below.

```
SELECT F_NAME, L_NAME
FROM EMPLOYEES
WHERE ADDRESS LIKE '%Elgin,IL%';
```

Upon execution, the query output should appear as shown below:

The screenshot shows a database IDE with a toolbar at the top containing icons for Browse, Structure, SQL, Search, Insert, and Execute. The SQL editor on the left contains the following query:

```
8
9 SELECT F_NAME , L_NAME
10 FROM EMPLOYEES
11 WHERE ADDRESS LIKE '%Elgin,IL%';
```

On the right, the 'Extra options' tab is active, displaying a table of results with columns F_NAME and L_NAME. Each row includes checkboxes for Edit, Copy, and Delete actions.

	F_NAME	L_NAME
☐ Edit ☐ Copy ☐ Delete	Alice	James
☐ Edit ☐ Copy ☐ Delete	Nancy	Allen
☐ Edit ☐ Copy ☐ Delete	Ann	Jacob

Now assume that you want to identify the employees who were born during the 70s. The query above can be modified to:

```
SELECT F_NAME, L_NAME
FROM EMPLOYEES
WHERE B_DATE LIKE '197%';
```

The output for this query will be:

The screenshot shows the same database IDE with the modified query in the SQL editor:

```
SELECT F_NAME , L_NAME
FROM EMPLOYEES
WHERE B_DATE LIKE '197%';
```

The results table on the right now displays four rows of employee data.

	F_NAME	L_NAME
☐ Edit ☐ Copy ☐ Delete	John	Thomas
☐ Edit ☐ Copy ☐ Delete	Alice	James
☐ Edit ☐ Copy ☐ Delete	Nancy	Allen
☐ Edit ☐ Copy ☐ Delete	Mary	Thomas

Note that in the first example, % sign is used both before and after the required text. This is to indicate, that the address string can have more characters, both before and after, the required text.

In the second example, since the date of birth in Employees records starts with the birth year, the % sign is applied after 197%, indicating that the birth year can be anything between 1970 to 1979. Further the % sign also allows any possible date throughout the selected years.

Consider a more specific example. Let us retrieve all employee records in department 5 where salary is between 60000 and 70000. The query that will be used is

```
SELECT *
FROM EMPLOYEES
WHERE (SALARY BETWEEN 60000 AND 70000) AND DEP_ID = 5;
```

Output for the query can be seen in the image below.

Server: phpMyAdmin demo - MySQL » Database: HR » Table: EMPLOYEES

Browse

Structure

SQL

Search

Insert

Export

Run SQL query/queries on table HR.EMPLOYEES:

1

2

3

4

SELECT *
FROM EMPLOYEES
WHERE (SALARY BETWEEN 60000 AND 70000) AND DEP_ID = 5;

	EMP_ID	F_NAME	L_NAME	SSN	B_DATE	SEX	ADDRESS	JOB_ID	SALARY	MANAGER_ID	DEP_ID
Edit Copy Delete	E1004	Santosh	Kumar	123459	1985-07-20	M	511 Aurora Av, Aurora, IL	400	60000.00	30004	5
Edit Copy Delete	E1010	Ann	Jacob	123415	1982-03-30	F	111 Britany Springs, Elgin, IL	220	70000.00	30004	5

☐ Check all

With selected:

Edit

Copy

Delete

Export

Sorting

You can sort the retrieved entries on the basis of one or more parameters.

First, assume that you have to retrieve a list of employees ordered by department ID.

Sorting is done using the ORDER BY clause in your SQL query. By default, the ORDER BY clause sorts the records in ascending order.

```
SELECT F_NAME, L_NAME, DEP_ID
FROM EMPLOYEES
ORDER BY DEP_ID;
```

The output for this query will be as shown below.

```
1 SELECT F_NAME, L_NAME, DEP_ID
2 FROM EMPLOYEES
3 ORDER BY DEP_ID;
```

☐	 Edit	 Copy	 Delete	John	Thomas	2
☐	 Edit	 Copy	 Delete	Ahmed	Hussain	2
☐	 Edit	 Copy	 Delete	Nancy	Allen	2
☐	 Edit	 Copy	 Delete	Alice	James	5
☐	 Edit	 Copy	 Delete	Steve	Wells	5
☐	 Edit	 Copy	 Delete	Santosh	Kumar	5
☐	 Edit	 Copy	 Delete	Ann	Jacob	5
☐	 Edit	 Copy	 Delete	Mary	Thomas	7
☐	 Edit	 Copy	 Delete	Bharath	Gupta	7
☐	 Edit	 Copy	 Delete	Andrea	Jones	7

Now, get the output of the same query in descending order of department ID, and within each deaprtment, the records should be ordered in descending alphabetical order by last name. For descending order, you can make use of the DESC clause.

```
SELECT F_NAME, L_NAME, DEP_ID
FROM EMPLOYEES
ORDER BY DEP_ID DESC, L_NAME DESC;
```

The output will be as shown in the image below.

Browse
 Structure
 SQL
 Search
 Insert
 Export
 Import

Run SQL query/queries on table HR.EMPLOYEES:

```

1 SELECT F_NAME, L_NAME, DEP_ID
2 FROM EMPLOYEES
3 ORDER BY DEP_ID DESC, L_NAME DESC;
  
```

						F_NAME	
☐		Edit		Copy		Delete	Mary
☐		Edit		Copy		Delete	Andre
☐		Edit		Copy		Delete	Bharat
☐		Edit		Copy		Delete	Steve
☐		Edit		Copy		Delete	Santos
☐		Edit		Copy		Delete	Alice
☐		Edit		Copy		Delete	Ann
☐		Edit		Copy		Delete	John
☐		Edit		Copy		Delete	Ahmed
☐		Edit		Copy		Delete	Nancy

Grouping

In this exercise, you will go through some SQL problems on Grouping.

NOTE: The SQL problems in this exercise involve usage of SQL Aggregate functions AVG and COUNT. COUNT has been covered earlier. AVG is a function that can be used to calculate the Average or Mean of all values of a specified column in the result set. For example, to retrieve the average salary for all employees in the EMPLOYEES table, issue the query: `SELECT AVG(SALARY) FROM EMPLOYEES;`

A good example of grouping would be if For each department ID, we wish to retrieve the number of employees in the department.

```

SELECT DEP_ID, COUNT(*)
FROM EMPLOYEES
GROUP BY DEP_ID;
  
```

```
1 SELECT DEP_ID, COUNT(*)
2 FROM EMPLOYEES
3 GROUP BY DEP_ID;
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain](#)]☐ Show all | Number of rows: 2

Extra options

DEP_ID	COUNT(*)
2	3
5	4
7	3

☐ Show all | Number of rows: 2

Now, for each department, retrieve the number of employees in the department and the average employee salary in the department. For this, you can use COUNT(*) to retrieve the total count of a column, and AVG() function to compute average salaries, and then GROUP BY.

```
SELECT DEP_ID, COUNT(*), AVG(SALARY)
FROM EMPLOYEES
GROUP BY DEP_ID;
```

```
SELECT DEP_ID, COUNT(*), AVG(SALARY)
FROM EMPLOYEES
GROUP BY DEP_ID;
```

Extra options

DEP_ID	COUNT(*)	AVG(SALARY)
2	3	86666.666667
5	4	65000.000000
7	3	66666.666667

You can refine your output by using appropriate labels for the columns of data retrieved. Label the computed columns in the result set of the last problem as NUM_EMPLOYEES and AVG_SALARY.

```
SELECT DEP_ID, COUNT(*) AS "NUM_EMPLOYEES", AVG(SALARY) AS "AVG_SALARY"
FROM EMPLOYEES
GROUP BY DEP_ID;
```

```

1
2 SELECT DEP_ID, COUNT(*) AS "NUM_EMPLOYEES", AVG(SALARY) AS "AVG_SALARY"
3 FROM EMPLOYEES
4 GROUP BY DEP_ID;

```

☐ Show all | Number of rows: 25 ▾

Extra options

DEP_ID	NUM_EMPLOYEES	AVG_SALARY
2	3	86666.666667
5	4	65000.000000
7	3	66666.666667

You can also combine the usage of GROUP BY and ORDER BY statements to sort the output of each group in accordance with a specific parameter. It is important to note that in such a case, ORDER BY clause must be used after the GROUP BY clause. For example, we can sort the result of the previous query by average salary. The SQL query would thus become

```

SELECT DEP_ID, COUNT(*) AS "NUM_EMPLOYEES", AVG(SALARY) AS "AVG_SALARY"
FROM EMPLOYEES
GROUP BY DEP_ID
ORDER BY AVG_SALARY;

```

The output of the query should look like:

```

1 SELECT DEP_ID, COUNT(*) AS "NUM_EMPLOYEES", AVG(SALARY) AS "AVG_SALARY"
2 FROM EMPLOYEES
3 GROUP BY DEP_ID
4 ORDER BY AVG_SALARY;

```

☐ Show all | Number of rows: 25 ▾

Extra options

DEP_ID	NUM_EMPLOYEES	AVG_SALARY
5	4	65000.000000
7	3	66666.666667
2	3	86666.666667

In case you need to filter a grouped response, you have to use the HAVING clause. In the previous example, if we wish to limit the result to departments with fewer than 4 employees, We will have to use HAVING after the GROUP BY, and use the count() function in the HAVING clause instead of the column label.

```

SELECT DEP_ID, COUNT(*) AS "NUM_EMPLOYEES", AVG(SALARY) AS "AVG_SALARY"
FROM EMPLOYEES
GROUP BY DEP_ID
HAVING count(*) < 4
ORDER BY AVG_SALARY;

```

```
1
2 SELECT DEP_ID, COUNT(*) AS "NUM_EMPLOYEES", AVG(SALARY) AS "AVG_SALARY"
3 FROM EMPLOYEES
4 GROUP BY DEP_ID
5 HAVING count(*) < 4
6 ORDER BY AVG_SALARY;
```

☐ Show all | Number of rows: 25 ▼

Extra options

DEP_ID	NUM_EMPLOYEES	AVG_SALARY
7	3	66666
2	3	86666

Practice Questions

1. Retrieve the list of all employees, first and last names, whose first names start with 'S'.

▼ Click here for Solution

```
SELECT F_NAME, L_NAME
FROM EMPLOYEES
WHERE F_NAME LIKE 'S%';
```

2. Arrange all the records of the EMPLOYEES table in ascending order of the date of birth.

▼ Click here for Solution

```
SELECT *
FROM EMPLOYEES
ORDER BY B_DATE;
```

3. Group the records in terms of the department IDs and filter them of ones that have average salary more than or equal to 60000. Display the department ID and the average salary.

▼ Click here for Solution

```
SELECT DEP_ID, AVG(SALARY)
FROM EMPLOYEES
GROUP BY DEP_ID
HAVING AVG(SALARY) >= 60000;
```

4. For the problem above, sort the results for each group in descending order of average salary.

▼ Click here for Solution

```
SELECT DEP_ID, AVG(SALARY)
FROM EMPLOYEES
GROUP BY DEP_ID
HAVING AVG(SALARY) >= 60000
ORDER BY AVG(SALARY) DESC;
```


Conclusion

Congratulations! You have completed this lab.

By the end of this lab, you are able to:

- Use string patterns for filtering the data retrieved.
- Sort the data retrieved upon one or more parameters using ORDER BY statement.
- Group the data with respect to a parameter.

Author(s)

[Abhishek Gagneja](#)

[Lakshmi Holla](#)

[Malika Singla](#)



Skills Network

© IBM Corporation 2023. All rights reserved.